

Báo cáo rà soát test code

<!-- framework: Selenium | run_id: run_20260620_170912_345854 -->

Framework: Selenium | Run ID: run_20260620_170912_345854

Tóm tắt kỹ thuật

- Framework: Selenium (Python)
- Kết quả chạy: 18 passed, không lỗi cú pháp hay import.
- Coverage: Không hỗ trợ coverage, nhưng các test hiện có bao phủ phần lớn các nhánh logic tính discount, shipping fee và cấu trúc form.
- Issue type: Không có lỗi thực thi nghiêm trọng; các test đều chạy thành công.
- Blocker quan trọng nhất: Thiếu một số trường hợp biên/ngoại lệ ($\text{weight} \leq 0$, $\text{fragile} = \text{true}$, $\text{credit} > 0$, và kiểm tra `event.preventDefault()`), gây lỗi hổng trong độ bao phủ yêu cầu.
- Artifact được review: `test_generated.py` (test script) so với `source_under_test.html` (source HTML/JS).

Lỗi nghiêm trọng

Không phát hiện.

Test còn thiếu

- Thiếu kiểm tra lỗi $\text{weight} \leq 0$ (yêu cầu “Nếu $\text{weight} \leq 0$ thì ném lỗi”).
- Bằng chứng: Không có test nào thiết lập `weight` bằng 0 hoặc âm và kiểm tra `total` không thay đổi hoặc thông báo lỗi.
- Thiếu kiểm tra khi `fragile` được chọn (phí cộng thêm 7.5).
- Bằng chứng: Các test `shipping fee` chỉ thay đổi `weight` và `destination`, không có trường hợp `fragile = true`.
- Thiếu kiểm tra khi $\text{weight} > 20$ (phí cộng thêm 25).
- Bằng chứng: Không có test nào thiết lập $\text{weight} > 20$ để xác nhận cộng thêm 25.
- Thiếu kiểm tra ảnh hưởng của `credit` (giảm tổng).
- Bằng chứng: Tất cả các test đều để `credit = 0`, không có trường hợp $\text{credit} > 0$ để xác nhận phép trừ.
- Thiếu kiểm tra `event.preventDefault()` (yêu cầu “Khi submit form thì `event.preventDefault()` được gọi”).
- Bằng chứng: Không có cách nào trong Selenium để trực tiếp xác nhận việc gọi `preventDefault`; test hiện không có bất kỳ kiểm tra nào liên quan.

Assertion yếu

Không phát hiện.

Rủi ro maintainability

- Phụ thuộc vào trạng thái DOM thay đổi trong runtime: Một số test (ví dụ `test_TC_002_calculateDiscount_unsupported_tier`) thêm option mới vào `<select>` bằng `execute_script`. Nếu các test được chạy song song hoặc trong môi trường không được reset hoàn toàn giữa các test, option “gold” có thể còn lại và gây ảnh hưởng tới các test sau.
- Hard-coded giá trị tính toán: Các giá trị tổng (“465.00”, “485.00” ...) được tính thủ công trong test. Nếu logic tính toán trong source thay đổi (ví dụ thay đổi phí cơ bản), các test sẽ cần cập nhật lại toàn bộ các giá trị này.

Gợi ý sửa ngay

1. Thêm test cho $\text{weight} \leq 0$

```
def test_TC_003_shippingFee_weight_zero_error(driver):
driver.get((Path.cwd() / 'source_under_test.html').resolve().as_uri())
driver.find_element(By.ID, "weight").clear()
driver.find_element(By.ID, "weight").send_keys("0")
driver.find_element(By.ID, "calculate").click()
total = driver.find_element(By.ID, "total")
assert total.text == "0.00"
```

2. Thêm test cho fragile = true

```
def test_TC_003_shippingFee_fragile_extra(driver):
driver.get((Path.cwd() / 'source_under_test.html').resolve().as_uri())
driver.find_element(By.ID, "weight").clear()
driver.find_element(By.ID, "weight").send_keys("6") # >5 => +10
Select(driver.find_element(By.ID, "destination")).select_by_value("domestic")
driver.find_element(By.ID, "fragile").click() # check
driver.find_element(By.ID, "calculate").click()
total = driver.find_element(By.ID, "total")
# subtotal 100, discount 0, shipping 5+10+7.5=22.5, total = 122.5
assert total.text == "122.50"
```

3. Thêm test cho weight > 20

```
def test_TC_003_shippingFee_weight_over_20(driver):
driver.get((Path.cwd() / 'source_under_test.html').resolve().as_uri())
driver.find_element(By.ID, "weight").clear()
driver.find_element(By.ID, "weight").send_keys("25")
Select(driver.find_element(By.ID, "destination")).select_by_value("domestic")
driver.find_element(By.ID, "calculate").click()
total = driver.find_element(By.ID, "total")
# shipping = 5 + 25 = 30
assert total.text == "135.00"
```

4. Thêm test cho credit > 0

```
def test_TC_004_credit_deduction(driver):
driver.get((Path.cwd() / 'source_under_test.html').resolve().as_uri())
driver.find_element(By.ID, "credit").clear()
driver.find_element(By.ID, "credit").send_keys("20")
driver.find_element(By.ID, "calculate").click()
total = driver.find_element(By.ID, "total")
# subtotal 100, discount 0, shipping 15, credit 20 => total = 95
assert total.text == "95.00"
```

5. Cải thiện isolation giữa các test

- Thêm fixture `@pytest.fixture(autouse=True)` để reload trang (`driver.refresh()`) trước mỗi test, hoặc sử dụng `driver.get(...)` ở đầu mỗi test (đã có) nhưng đồng thời xóa các option tùy chỉnh được thêm bằng script:

```
@pytest.fixture(autouse=True)
```

```
def clean_up(driver):
```

```
yield
```

```
driver.execute_script("var sel = document.getElementById('tier');
```

```
sel.querySelectorAll('option[value=\\"gold\\"]').forEach(o=>o.remove());")
```

6. Ghi chú về `event.preventDefault()`

- Vì Selenium không thể trực tiếp xác nhận việc gọi `preventDefault`, có thể kiểm tra rằng URL không thay đổi sau khi click nút submit (điều này ngụ ý form không thực hiện navigation). Thêm một test:

```
def test_TC_004_prevent_default(driver):
```

```
driver.get(Path.cwd() / 'source_under_test.html').resolve().as_uri())
current_url = driver.current_url
driver.find_element(By.ID, "calculate").click()
assert driver.current_url == current_url
```

Áp dụng các sửa đổi trên sẽ nâng cao độ bao phủ, giảm rủi ro maintainability và đáp ứng đầy đủ các yêu cầu trong Requirement JSON.